

Hosting the AI SDR Dashboard

What it would take to reach the app from somewhere other than the machine it runs on — and what that decision actually turns on.

The one thing that decides everything

QuickMail's published API cannot read a reply, send an email, mark a prospect do-not-contact, or take a campaign live. We do all four through QuickMail's own web app, driven by a signed-in browser on the operator's desk.

That browser is the constraint. **Seven of the app's routes need it. Three run on the API key alone.** Everything else — the dashboard, 50,547 leads, search, templates, the block list, AI drafting — is local and would host anywhere without complaint.

FEATURE	NEEDS THE BROWSER?	WHY
Sync replies	yes	Reply content exists only in QuickMail's internal API.
Send a reply	yes	No send operation on any public API.
Stop a prospect	yes	Do-not-contact is not exposed publicly.
Block a domain	yes	Same — internal API only.
Approve a stop	yes	Pushes the block through to QuickMail.
Forward to manager	yes	Sends using QuickMail's own mailbox authorisation.
Campaign go-live	yes	No trigger field exists; the UI is clicked.
Create a campaign	no	Public v2 API, API key only.
Enrol leads	no	Public v2 API.
Stats sync	no	Public v2 API.
Everything else	no	Reads the local database. No QuickMail involvement.

So "can we host it?" is really "what happens to those seven?" Hosting the app itself is straightforward. Hosting the browser session is the question, and it has no clean answer — which is why the options below trade off rather than solve.

Why the browser cannot simply move to a server

The obvious idea — run a headless browser on the server, sign it into QuickMail once — runs into two problems that are not engineering effort but design:

- **The login is "Continue with Google".** Automating an OAuth flow with two-factor authentication is possible but brittle, and it means storing credentials that can reach far more than QuickMail.
- **Session expiry fails silently.** When the stored session lapses, sync simply stops returning replies. No error, no bounce — the queue just goes quiet, which is the failure mode this project has spent most of its effort designing against.

The five options

Ordered by how much has to change.

	OPTION	EFFORT	OPERATOR'S PC ON?	WHAT YOU GET
1	Tunnel Tailscale or Cloudflare Tunnel	~1 hour	yes	A reachable URL pointing at the app where it already runs. Everything works — the browser is right there. Nothing is rewritten.
2	Split hosted UI, local worker	2–3 days	yes	App and database hosted properly; a small worker on the operator's PC picks up browser jobs from a queue. Everything works , and the UI is always up even when the PC is not.
3	Full host headless browser on the server	2–3 days + ongoing	no	Truly independent — but depends on automating a Google login and keeping a session alive. Not recommended ; see page 1.
4	Replace the dependency IMAP + SMTP	~1 week	no	Read replies over IMAP and send over SMTP, straight from the 17 mailboxes. Needs an app password for each. Still leaves two features stranded — do-not-contact and go-live have no API at all.
5	Email only no app access	done	no	Replies are already forwarded to the manager's inbox. They read there and reply, or hand it back. No hosting, no exposure, nothing to maintain.

The recommendation

TODAY

Option 5 — already running

- The stated requirement was "forward all the real replies to the manager".
- That works now: real replies go to their inbox automatically, auto-responders filtered out.
- No hosting, no exposure, no new credentials.

WHEN THEY NEED TO CLICK INTO IT

Option 1 — Tailscale

- Install on both machines; the manager reaches a private address.
- No firewall rules, no public exposure, encrypted by default.
- Free at this size, about an hour, and nothing is rewritten.

Option 2 becomes worth its cost only when "the operator's PC must be on" stops being acceptable — a real constraint eventually, but not one that exists today. Options 3 and 4 solve a problem nobody currently has, and neither solves it completely.

Worth saying plainly: if the manager is going to work out of their inbox anyway, hosting buys very little. The app's advantage is the AI draft, the permanent block list, and the audit trail — everything else they can do from the forwarded email.

Before anything is exposed

These apply to every option above. They are not hosting work — they are the cost of the app being reachable by anyone other than the person sitting at it.

ITEM	WHY IT MATTERS, AND THE FIX
1 The password is still the shipped default	<code>changeme123</code> guards 50,547 real people's names, emails and phone numbers. Thirty seconds to fix: <code>npm run set-password -- admin@sybrant.com</code>
2 The seed would restore it	<code>SEED_ADMIN_PASSWORD</code> is still in <code>.env</code> , so re-seeding puts the default back. Blank it once step 1 is done.
3 No limit on password attempts	Nothing slows repeated guesses. On a private tunnel the network is the gate; on anything public this has to be added first.
4 Writes to QuickMail are live	<code>QUICKMAIL_DRY_RUN=false</code> . Anyone who gets in can send email as your sending domains.
5 31 MB of personal data in a single file	Fine on one machine. On a server it needs encryption at rest, a backup plan, and it is GDPR-relevant — names, addresses, phone numbers and the content of people's replies.

One thing already fixed, worth knowing

Access was previously guarded only by whether a session *cookie existed*, with nothing checking it further. A cookie containing arbitrary text was enough to read every lead and every reply. That is now checked where the data is read, and forged or expired sessions land on the sign-in page. It was found while investigating a missing sidebar.

Running it on the local network today

Reaching the app from another machine on the same office network needs no hosting decision at all:

- **Use a production build.** `npm run build` then `npx next start -H 0.0.0.0`. The development server serves its live-reload socket at `localhost`, so a browser on another machine never finishes loading the page and the sign-in button silently does nothing.
- **Set `AUTH_TRUST_HOST=true`.** Without it, signing in over an IP address fails while `localhost` keeps working.
- **Allow the port inbound** for the private network only.
- **Browse to the machine's IP**, not `localhost` — on another machine that means *that* machine.

Do item 1 first. It takes thirty seconds and matters more than which hosting option is chosen. Everything else on this page can wait for a decision; that one cannot.

AI SDR Dashboard · prepared for the team · figures read from the live account and the local database.